

Lerna Protocol

Factory Rights Settlement Profiles for Hydra/Cardano eUTxO

Arthur Zhang

8 June 2026

Abstract

Lerna is a Hydra-native discipline for representing many virtual-channel rights compactly inside a Hydra Head or an Interhead virtual Head, and for deterministically materialising those rights when optimistic updating stops. It is not a replacement for Hydra close, contest, or fanout. It sits beneath those mechanisms and supplies the missing inner rule: compact factory rights must be turned into ordinary Head UTxOs or claim UTxOs before they can be safely exposed to the outer settlement layer. The paper presents one protocol with two settlement profiles. The Head-local factory profile keeps factory rights inside the Head ledger and is safe only when the latest Lerna state is embedded in signed snapshot evidence before close or available for contest. The claim-materialisation profile makes that repayment explicit through a ledger-enforced claim object, a deterministic transition relation, bounded admission rules, verifiable evidence binding, and a conditional acceptance theorem.

The design is intentionally bounded. Lerna does not compete with Interhead Hydra’s virtualisation of Heads across Heads, nor with Hydrozoa’s broader execution/custody separation and deterministic outer rollout. Lerna uses those works as boundary conditions: Interhead explains where virtual Heads may live and why enforceable state must survive failed cooperation; Hydrozoa shows why compact off-chain state must have deterministic payout order, fallback discipline, and no stranded value when custody is at stake. Hydra Head supplies the close/contest/fanout substrate and the practical pressure that makes compactness valuable. Lerna’s originality is narrower and more internal: an inner factory-rights discipline with explicit host-interface obligations, deterministic materialisation, non-interference, release receipts, balance preservation, and no-stranding. The claim-materialisation profile is conditional on Hydra contestability, ledger validation of submitted transactions, proof availability, watcher/operator liveness, and materialisation capacity inside the committed budget. If the Head-local liveness assumption is unacceptable, the claim-materialisation profile is the appropriate safety envelope.

Keywords: Cardano, Hydra, eUTxO, claim UTxO, virtual channels, state channels, non-interference, native assets, release receipts, materialisation.

Contents

1	Introduction	1
2	Problem Statement and Design Thesis	2
2.1	The Compactness Debt Problem	2
2.2	Design Thesis	3
2.3	Lessons from the Host References	3
2.4	Profile Boundary	3
3	Layering and Positioning	3
3.1	Lerna below Interhead: from virtual Heads to virtual rights	3
3.2	Inner rollout before outer rollout: Lerna and Hydrozoa	4
3.3	Why compact factory state matters for Hydra fanout	5
3.4	Compatibility across host ledgers	6
3.5	What Lerna deliberately does not take from each prior work	6
4	System and Threat Model	7
4.1	Host Interface	7
4.2	Actors	7
4.3	Adversary	8
4.4	Assumptions	8
5	Factory and Claim State	8
6	Protocol State Machine	10
7	Materialisation Transition	10
7.1	Admission	10
7.2	Authorisation	11
7.3	Conservation	11
7.4	Release Receipts	12
7.5	Continuation	12
8	Ledger Enforcement	12
9	Acceptance Argument for the Claim-Materialisation Profile	13
10	Evidence and Evaluation	15
10.1	Evaluation Targets	16
11	Related Work and Positioning	16
12	Limitations and Open Questions	16
12.1	Open Questions	17
13	Conclusion	18
A	Protocol Review Matrix	18

1 Introduction

Hydra Head gives a fixed participant set a Cardano-like off-chain ledger: participants deposit UTxOs, sign snapshots, close with a snapshot when necessary, contest stale closes, and fan out the resolved UTxO set to L1 [1, 2]. This is already a powerful settlement substrate. A virtual-channel factory layered inside or near a Head should therefore avoid inventing an unmeasured parallel dispute game. Its hard problem is narrower and more operational: when compact rights are no longer being updated optimistically, what exact inner object or transition discipline turns them back into ordinary ledger claims without double release, stale-state settlement, or silent asset loss?

Lerna answers that question as a factory-rights protocol with two safety envelopes. In the Head-local factory profile, compact rights remain inside the Head or virtual Head and are safe only if the latest Lerna state is already embedded in a signed Hydra snapshot before close, or if an honest actor can contest a stale close with such a newer signed snapshot. In the claim-materialisation profile, the same repayment is made explicit by a ledger-controlled claim object carrying unresolved value, latest-state metadata, authorisation policy, release-receipt state, proof-availability commitments, and materialisation bounds. A final settlement consumes all unresolved claim value. A partial settlement creates a continuation claim with a strictly advanced settled-right frontier.

This paper keeps those two profiles in one theory without ordering them as a delivery story. Claim-materialisation is not presented as a Hydra replacement, a Hydrozoa competitor, or an “eltoo for Cardano”. It is the stronger safety envelope for deployments that cannot rely solely on Head-local snapshot availability. Instantiation evidence is useful as corroboration, but the thesis is stated at the level of ledger objects, transition invariants, and acceptance conditions rather than at the level of a particular codebase.

Contributions. The paper makes four protocol contributions.

C1. Inner factory rights discipline.

Lerna defines the compact object that lives below Hydra fanout: a factory-local rights state binding factory identity, Hydra Head identity, network id, settlement epoch, latest-state number, unresolved value, signer policy, release-receipt root, proof-availability root, and materialisation bounds. The same discipline can be represented as Head UTxOs in the Head-local profile or as a claim datum in the claim-materialisation profile (Section 5).

C2. Deterministic inner materialisation before outer fanout or rollout.

Lerna makes compactness payable. Before the outer Hydra fanout, Interhead movement, or Hydrozoa-style rollout is relied upon for custody, compact factory rights must be deterministically expanded into ordinary ledger outputs or into a continuation object with an explicitly bounded remaining value (Sections 7 and 8).

C3. Factory-local non-interference plus release receipts.

Lerna separates authorised partial settlement from unauthorised interference. Reduced-signature materialisation is admissible only with a factory-local frame proof that untouched owners remain untouched. Each released right also receives a deterministic receipt, so the protocol has both a non-interference argument and a nullifier against double release (Definitions 9.3 and 9.4).

C4. Host-interface and no-stranding obligations.

Lerna states what it needs from the surrounding Head or ledger without turning that host into part of the Lerna protocol: scoped identities, authenticated state

evidence, output accounting, validity deadlines, bounded transaction capacity, and a way to submit admissible materialisation. Under those obligations, an accepted transition either pays selected rights or preserves the remaining value in one bounded continuation; it does not leave compact rights in an unreachable or ambiguous state (Section 4.1 and definition 9.5).

Claim status. This document is a protocol thesis, not a complete proof of every deployment setting. **Definitions** specify the intended ledger objects and transition interfaces. **Assumptions** state the external conditions Lerna does not force by itself. **Lemmas** state local safety facts under those assumptions. **Definition 9.6** is a narrow acceptance theorem for the claim-materialisation profile only; it should not be read as a theorem for arbitrary Head-local liveness, arbitrary application semantics, or cross-Head composition.

Design principles.

- P1. **Compactness is debt.** A compact factory object is admissible only because the protocol defines how it becomes ordinary ledger claims again.
- P2. **Outer settlement is not inner materialisation.** Hydra fanout, Interhead movement, and Hydrozoa-style rollout should see materialised rights or a bounded continuation, not an opaque unresolved factory.
- P3. **Same-scope supersession only.** Latest-state replacement is valid only inside one domain, network, Head, factory, and settlement epoch.
- P4. **Conservation is exact.** Lovelace and native assets must be conserved across selected outputs and any continuation.
- P5. **Locality is a proof obligation.** Reduced-signature settlement is safe only when the touched rights and untouched frame are explicit.
- P6. **Receipts are nullifiers.** Every materialised right advances a receipt root and frontier so the same right prefix cannot be settled twice.
- P7. **Assumptions are first-class.** Proof availability, watcher/operator liveness, and capacity bounds must be stated where the protocol cannot enforce them.
- P8. **Fallback preserves enforceable state.** If cooperation stops, the protocol must move either to signed Head evidence or to a ledger-visible claim; it may not strand rights in an opaque local record.
- P9. **Deterministic order beats discretionary payout.** Multi-batch materialisation must use canonical ordering and a monotone frontier so the next claim state is independent of operator preference.

2 Problem Statement and Design Thesis

2.1 The Compactness Debt Problem

The central problem is not how to make a Hydra Head hold fewer UTXOs. That is only the local optimisation. The harder problem is how to make the compression honest at the moment when cooperative updating stops. A compact factory object stands for many rights, owners, release conditions, and asset claims. If that object reaches close, fanout, or outer rollout without a deterministic materialisation discipline, the host ledger sees only a small object while users expect individual claims.

Lerna treats this as a compactness debt. The debt is paid when the compact factory state is expanded into ordinary ledger outputs, release receipts, and, where necessary, one bounded continuation. This framing explains why Section 5 defines the compact state before Section 7 defines materialisation, and why Section 9 proves only the claim-materialisation envelope rather than the whole family of possible host systems.

2.2 Design Thesis

The central thesis is:

A Hydra-native factory is safe only when compact rights are treated as ledger debt that must be deterministically materialised under explicit snapshot, evidence, and capacity assumptions.

This thesis separates three questions that are often collapsed. First, where is the latest compact factory state authenticated? Second, who is authorised to materialise which rights? Third, what exact ledger value must appear after materialisation? The Head-local profile answers the first question through signed Head snapshots. The claim-materialisation profile answers it through a claim object at the ledger boundary. Both profiles rely on the same inner discipline: same-scope supersession, exact conservation, non-interference for partial settlement, release receipts, and bounded continuation.

2.3 Lessons from the Host References

The two host references used by this thesis point to the same design demand from different directions. Interhead treats virtual Head state as enforceable state that must survive failed cooperation by moving into a ledger-enforced path when necessary [5]. Hydrozoa separates ordinary cooperative operation from a fallback regime and gives deterministic rollout rules so large L2 withdrawal sets are paid without stranding value [6]. Lerna adapts those lessons inward: it does not virtualise Heads and it does not define a full L2 ledger, but it does require compact factory rights to remain enforceable, ordered, and payable.

2.4 Profile Boundary

Table 2 and section 3 position these envelopes relative to Hydra, Interhead, and Hydrozoa. Section 4 fixes the adversary and assumptions. Sections 5 to 8 define the protocol objects, state machine, and checks. Section 9 gives the narrow acceptance argument, while Sections A and 10 to 12 state how the thesis should be reviewed.

3 Layering and Positioning

Lerna stays inside the Cardano/Hydra settlement vocabulary. Cardano L1 supplies eUTxO accounting, script-governed outputs, native assets, validity intervals, and min-ADA rules [4]. Hydra supplies the outer Head lifecycle: open, snapshot, close, contest, and fanout. Lerna supplies the inner factory discipline used when many compact virtual-channel rights must be materialised into ordinary ledger outputs. Figure 1 summarises this placement.

3.1 Lerna below Interhead: from virtual Heads to virtual rights

Interhead virtualises Hydra Heads across Heads: it studies how two Heads and their intermediaries can move a virtual Head state between Head contexts while preserving adjudication conditions [5]. Lerna operates at the next smaller granularity. It does not create a virtual Head and does not

Table 1: Host-reference lessons absorbed by Lerna.

Reference pattern	Lerna consequence	Where it appears
Enforceable virtual state	Compact rights are safe only if they can become ordinary claims under the same scope.	Settlement scope, state commitments, and claim admission.
Fallback after failed cooperation	The Head-local profile needs signed snapshot evidence; otherwise the claim path is the safety envelope.	Profile boundary and liveness assumptions.
Deterministic payout ordering	Partial materialisation uses canonical right order, release receipts, and a monotone frontier.	Materialisation transition and continuation rules.
No stranded value	Every accepted partial transition preserves unresolved value in exactly one continuation; final acceptance requires zero unresolved value.	No-stranding lemma and acceptance theorem.

specify cross-Head transport. Instead, it virtualises rights and channels inside one physical Head, or inside a virtual Head supplied by an Interhead construction.

The distinction is important. A virtual Head still eventually exposes a Head state to Hydra-like closing and settlement. Lerna asks what happens inside that state when a factory has compressed many bilateral or application-specific rights into a small number of Head UTxOs. The answer is factory-local: the latest Lerna state must bind the Head id, factory id, network id, settlement epoch, unresolved value, owners, receipts, and materialisation budget. That state may live as compact Head-local accounting, or it may be represented by a claim UTxO when explicit ledger settlement is required. In both cases, Lerna is below Interhead: Interhead says where the virtual Head may be; Lerna says how rights inside that Head are released.

3.2 Inner rollout before outer rollout: Lerna and Hydrozoa

Hydrozoa separates execution from custody and gives deterministic rollout or fallback procedures for turning L2 ledger effects into L1 transactions when large withdrawal sets must be paid out [6]. Lerna borrows the design ethic, not the ledger architecture. It treats compact factory rights as an inner state that must be deterministically rolled out before the outer system is asked to carry custody.

This creates a simple ordering rule. First, materialise the compact Lerna factory state into ordinary Head UTxOs or into a bounded continuation claim. Only then rely on Hydra fanout, Interhead movement, or Hydrozoa-style rollout for the outer layer. Without this inner rollout, the outer layer sees a small UTxO whose datum names a compressed factory, not the individual rights that users expect to receive. Lerna’s claim-materialisation profile instantiates this rule: it turns a compact claim into selected settlement outputs, release receipts, and, if necessary, one continuation claim whose remaining value and frontier are explicit.

Table 2: Settlement safety envelopes in Lerna.

Profile	Safety envelope	When to use it
Head-local factory	Compact state remains inside the Head ledger and is safe only when the latest factory state is in signed snapshot evidence before close or can be used for contest.	Use when signed snapshot availability and watcher liveness are acceptable assumptions.
Claim-materialisation	Compact state becomes a ledger-controlled claim object; settlement must satisfy scope, authorisation, conservation, receipts, continuation, and admission bounds.	Use when Head-local snapshot availability is too strong and explicit ledger settlement is required.

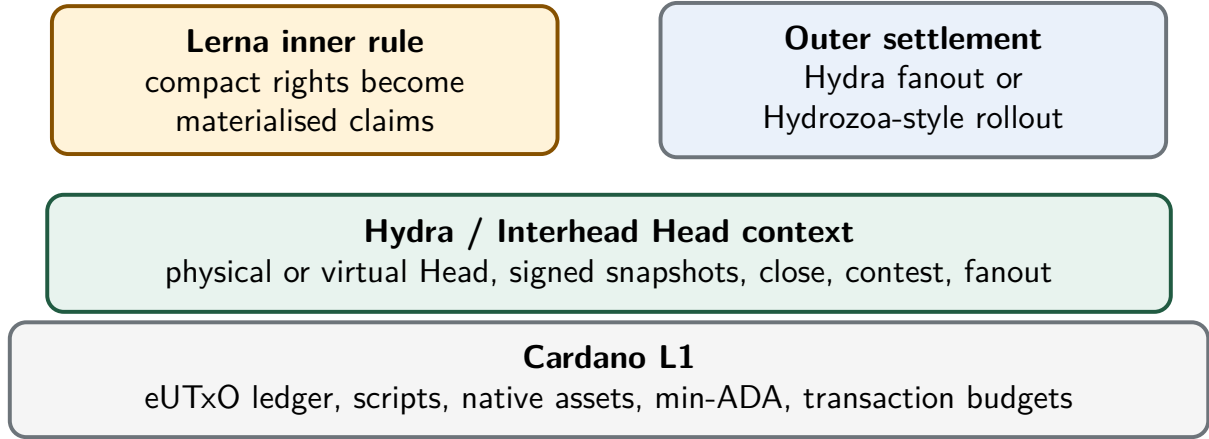


Figure 1: Lerna’s location in the stack. Hydra and Interhead provide Head contexts and the outer settlement lifecycle; Hydrozoa motivates deterministic rollout when custody is exposed. Lerna supplies the inner rule for compact factory rights inside a physical or virtual Head.

Hydrozoa mechanism	Lerna analogue
Settlement/finalisation pays as many withdrawals as fit, then leaves a rollout UTxO for the rest	Materialisation pays as many selected rights as fit, then leaves one continuation claim or compact factory continuation.
Deterministic withdrawal ordering	Canonical settled-right frontier and byte-ordered right ids.
Rollout beacon is consumed when rollout completes	Final Lerna acceptance requires no continuing claim and no unresolved lovelace or native assets.
Rule-based fallback after optimistic cooperation fails	Claim-materialisation profile when Head-local checkpoint or contest evidence cannot be relied upon.

3.3 Why compact factory state matters for Hydra fanout

Hydra fanout is the final exposure point of a Head state. The Hydra documentation records practical constraints around Head finalisation: large or complex UTxO sets can exceed fanout

capacity, and tokens minted but not burned inside an open Head can prevent finalisation [3]. These are not abstract inconveniences for factories. A naive virtual-channel factory that keeps every right as a separate Head UTxO can create fanout pressure exactly when the system is least able to negotiate new structure. A factory that mints control tokens internally and leaves them unresolved can create a finalisation hazard.

Lerna’s compactness is a direct response to that pressure. During ordinary operation, the Head carries a small factory object rather than a fanout-sized collection of user outputs. Compactness, however, is debt. Before the Head is closed, contested, fanned out, or rolled into another custody context, Lerna must repay the debt by deterministic materialisation: selected rights become ordinary outputs, release receipts advance, remaining rights are represented by one bounded continuation, and the value equation is checked exactly over lovelace and native assets. Compact state is therefore acceptable only because the protocol specifies how it becomes non-compact again.

The token rule is equally strict. If Lerna uses factory, receipt, or control tokens inside a Head, those tokens must either be burned before finalisation or be represented as ordinary conserved assets in the materialised state. Compact factory accounting is not allowed to hide a minted-token finalisation hazard.

3.4 Compatibility with physical Heads, Interhead virtual Heads, and Hydrozoa-style ledgers

Lerna is compatible with three host settings.

Physical Hydra Head.

The Head-local profile stores compact factory state directly in the Head ledger. The claim-materialisation profile introduces a claim UTxO when the factory needs script-controlled release, reduced-signature settlement, or multi-batch continuation before outer fanout.

Interhead virtual Head.

Interhead may supply the virtual Head context. Lerna does not alter the Interhead conversion or punish phases. It binds its own rights state to the virtual Head’s identity and requires inner materialisation before that state is treated as settled across Head boundaries.

Hydrozoa-style ledger.

A Hydrozoa-style system may provide deterministic outer rollout and rule-based fallback. Lerna can be used as an inner rights factory whose compact state is expanded into ledger outputs before the Hydrozoa rollout algorithm is asked to distribute custody.

The compatibility claim is deliberately modest. Lerna assumes the host system has a notion of ledger outputs, value conservation, domain separation, deadlines, and a way to submit deterministic materialisation transactions. It does not require the host system to adopt one particular claim script, and it does not require Lerna to adopt the host system’s full dispute game.

3.5 What Lerna deliberately does not take from each prior work

From Hydra Head.

Lerna takes the Head lifecycle as substrate. It does not replace open, close, contest, or fanout, and it cannot rescue a stale Head close that no authorised actor contests in time.

- From Interhead.** Lerna takes the idea that Head contexts can themselves be virtualised. It does not take over Interhead’s cross-Head conversion protocol or intermediary aggregation. Its unit is a right inside a Head, not a Head across Heads.
- From Hydrozoa.** Lerna takes the discipline of deterministic rollout when compact L2 state must be exposed to custody. It does not adopt Hydrozoa’s execution/custody split as its own ledger model and is not a competing state-channel architecture.
- From eltoo, factories, and Perun.** Lerna takes the lesson that latest-state replacement and factory adjudication should avoid punitive games when possible [7, 8, 9]. It does not claim to be an eltoo construction for Cardano, and it does not import Bitcoin-style update transactions into the eUTxO setting.

4 System and Threat Model

4.1 Host Interface

Lerna is intentionally specified below the host lifecycle. A host Head, virtual Head, or compatible ledger environment must provide the following abstract interface:

Scoped identity.

A stable domain, network, Head, factory, and epoch namespace for every compact state and materialisation witness.

Authenticated state evidence.

A way to recognise the latest same-scope state, either as signed Head evidence or as a ledger-visible claim datum and transition witness.

Ledger accounting.

Explicit outputs, values, native assets, validity intervals, and transaction budgets sufficient to check exact conservation and admission bounds.

Submission path.

At least one authorised path by which admissible materialisation, continuation, or final settlement can reach the host ledger within the relevant deadlines.

The host interface is a contract between the thesis and its environment. If an environment cannot provide one of these capabilities, Lerna may still be a useful local accounting convention, but the settlement safety claim in this paper does not apply.

4.2 Actors

Head participant.

A Hydra party able to run a Hydra node, sign snapshots, close, and contest.

Virtual participant.

An application endpoint whose right is represented in Lerna factory state. A virtual participant may or may not be a Head participant.

Operator.

An application actor that can coordinate batching, submit transactions, or fund ordinary fees. The operator does not gain authority to redirect factory value.

Watcher. An actor that observes Head lifecycle events and Lerna evidence, and can cause fresh evidence or settlement transactions to be submitted through an authorised path.

4.3 Adversary

The adversary may corrupt participants, operators, watchers, or Head participants; withhold signatures; provide stale latest-state evidence; propose same-number conflicting states; withhold proof bodies; try replay across domains, networks, Heads, factories, or epochs; create outputs below min-ADA; over-release lovelace or native assets; reorder multi-batch settlement; mutate continuation datums; leave minted control tokens unresolved; force large fanout sets; or tamper with evidence used for materialisation acceptance.

4.4 Assumptions

Assumption 4.1 (Hydra contestability). If the Head is closed with stale state, at least one honest actor can observe the close and contest with newer valid Head evidence before the contestation deadline.

Assumption 4.2 (Cardano ledger validation). Submitted materialisation, claim-creation, claim-settlement, and continuation transactions are validated by the Cardano ledger according to the applicable ledger rules.

Assumption 4.3 (Head-local checkpoint or materialisation). When the Head-local factory profile is used, safety requires at least one authorised Head participant, operator, or watcher to ensure that the latest Lerna state is represented in a signed Hydra snapshot before the Head is closed, or to contest a stale close with an already-signed newer snapshot before the relevant contestation period expires. Lerna does not assume that fresh Head transactions can be inserted after close. If this liveness and evidence availability assumption is unacceptable for the application or deployment environment, the design must use the claim-materialisation profile rather than relying on Head-local accounting alone.

Assumption 4.4 (Proof availability). Proof bodies committed by the proof witness root are available to the verifier before materialisation is accepted.

Assumption 4.5 (Watcher and operator liveness). At least one actor with sufficient evidence and authority can submit accepted materialisation plans within the relevant Head and ledger deadlines. This is a liveness assumption, not a claim that Lerna can force offline parties to act.

Assumption 4.6 (Bounded materialisation). The number of outputs, proof bytes, execution units, and continuation value fit within the budget committed in the factory state or claim datum.

5 Factory and Claim State

Definition 5.1 (Settlement scope). A Lerna settlement scope is the tuple

$$\Sigma = (\text{domain}, \text{networkId}, \text{hydraHeadId}, \text{factoryId}, \text{claimEpoch}).$$

Every latest-state header, transition, receipt, proof-availability commitment, and non-interference proof is interpreted only inside this scope.

Figure 2 shows how the two settlement profiles share this scope while relying on different enforcement envelopes.

The word `claimEpoch` is used here as a settlement epoch. In the Head-local profile, the same scope binds compact factory state inside the Head ledger. In the claim-materialisation profile, the scope is carried by the claim datum and checked at the ledger boundary.

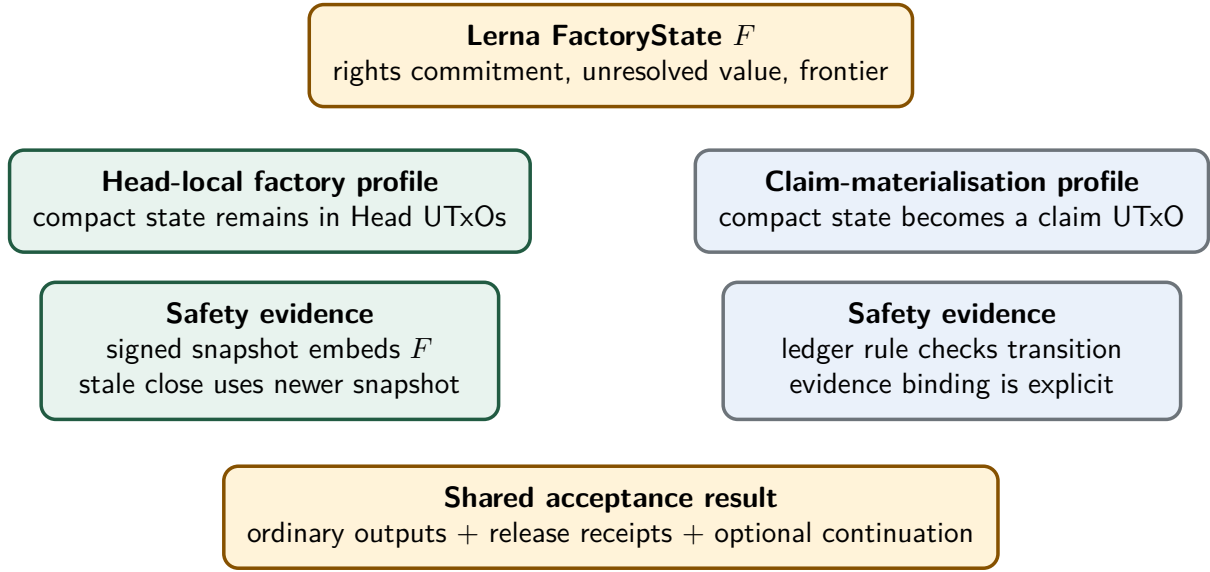


Figure 2: The two settlement profiles are safety envelopes for the same compact factory state. Head-local settlement is lighter but depends on signed snapshot availability. Claim-materialisation is heavier but makes the repayment of compact rights explicit at the ledger boundary.

Definition 5.2 (Factory state). A Lerna factory state F is a compact Head-local object:

$$F = (\Sigma, \text{latestStateNumber}, \text{rightsCommitment}, \text{unresolvedValue}, \text{signerPolicy}, \text{releaseReceiptRoot}, \text{settledRightFrontier}, \text{materialisationBudget}).$$

The rights commitment is a commitment to the ordered rights frame: each right’s owner, value, application channel id, and release condition. The unresolved value is exact over lovelace and native assets. The frontier is the greatest canonical right prefix already materialised or released.

Definition 5.3 (Head snapshot embedding). A Head-local factory state is embedded when a Hydra snapshot contains one or more Head UTxOs whose datum and value commit to F . A newer embedded factory state supersedes an older one only if it has the same settlement scope and a strictly greater latest-state number. A materialised embedding replaces the compact factory UTxO set by ordinary Head UTxOs for selected rights plus, if needed, one compact continuation whose frontier and unresolved value are advanced deterministically.

Lemma 5.4 (Head-local release safety). *Assume Hydra contestability, signed-snapshot availability, and bounded materialisation. If the closed or successfully contested Head snapshot embeds the greatest signed Lerna factory state F , then deterministic materialisation of F cannot release more value than $F.\text{unresolvedValue}$, cannot release a right at or below $F.\text{settledRightFrontier}$, and cannot change rights outside the authorised non-interference frame.*

Proof. The signed Hydra snapshot selects the enforceable Head UTxO set. Scope binding prevents replay of a factory state across domains, networks, Heads, factories, or settlement epochs. Strict latest-state supersession selects one factory state among same-scope candidates. Canonical frontier advancement excludes already-released rights. Exact value conservation requires selected outputs plus any compact continuation to equal $F.\text{unresolvedValue}$. The non-interference frame restricts reduced-signature materialisation to signed owners and preserves the untouched rights commitment. The lemma is conditional: if the greatest signed factory state is not available for close or contest, the Head-local profile does not provide this safety envelope. $\square$

The claim-materialisation profile uses the following domain separators:

Domain	Purpose
Lerna:ClaimUTx0:LatestStateHeader	latest-state and transition binding
Lerna:ClaimUTx0:ReleaseReceipt	release receipt nullifiers
Lerna:ClaimUTx0:ProofAvailability	proof witness package binding
Lerna:ClaimUTx0:NonInterferenceProof	reduced-signature frame proof

Definition 5.5 (Claim datum). A claim datum C is the ledger representation of compact factory rights in the claim-materialisation profile. It contains:

$C = (\text{factoryId}, \text{hydraHeadId}, \text{claimEpoch}, \text{networkId}, \text{domain}, \text{latestStateNumber}, \text{stateCommitment}, \text{releaseReceiptRoot}, \text{nonInterferenceRoot}, \text{authorisationMode}, \text{unresolvedLovelace}, \text{unresolvedAssets}, \text{minAdaFloor}, \text{requiredSigners}, \text{settledRightCount}, \text{lastSettledRightId}, \text{maxOutputCount}, \text{maxProofSizeBytes}, \text{maxExecutionMemory}, \text{maxExecutionSteps}, \text{proofAvailabilityMode}, \text{proofWitnessRoot}).$

The state commitment is recomputed from length-prefixed canonical fields, not accepted as an independent assertion.

The claim value is:

$$\text{Value}(C) = \text{lovelace}(C.\text{unresolvedLovelace}) + C.\text{unresolvedAssets}.$$

The Head-local profile uses the same conceptual fields, but their enforcement comes from the Head snapshot discipline and the authorised checkpoint/materialisation assumption rather than from spending a claim UTxO on L1. This is the main reason the paper treats the two settlement profiles as safety envelopes rather than as deployment stages.

6 Protocol State Machine

Lerna is easiest to audit as a small state machine around compactness debt. The state machine is not a new host-ledger lifecycle. It is the inner discipline that decides whether a compact factory state is still safely compact, has been checkpointed inside a Head, or must be materialised through a claim object.

Table 3 gives the operational reading of Definitions 5.2, 5.3 and 5.5. The transition section below specifies the claim-materialisation path in detail; the Head-local path remains conditional on signed snapshot availability.

7 Materialisation Transition

A materialisation transition selects a canonical batch of rights $R = (r_1, \dots, r_k)$ with matching settlement outputs $O = (o_1, \dots, o_k)$. In the claim-materialisation profile, the transition spends the current claim UTxO, provides a redeemer D , an authorised signer set S , and produces either a final zero-claim result or a continuation claim datum C' . In the Head-local profile, the same transition discipline must be reflected in the Head snapshot before the outer Hydra contestation window can be relied upon. Figure 3 gives the claim-materialisation view of the same transition.

7.1 Admission

The transition is admitted only if:

1. $D.\text{nextStateNumber} > C.\text{latestStateNumber}$.

Table 3: Lerna state-machine view.

State	Meaning	Admissible next step
Compact factory	Rights are represented by a compact state F inside a Head or virtual Head.	Continue updating, checkpoint F , or create a claim object before outer settlement is relied upon.
Head-local checkpoint	A signed Head snapshot embeds the latest same-scope factory state.	Use Head-local materialisation if Definition 4.3 holds, or move to claim-materialisation if it does not.
Claim-open	A claim datum C carries unresolved value, signer policy, receipt state, frontier, and bounds.	Apply an admissible materialisation transition under Section 7.
Partial materialisation	A selected canonical batch has been paid and one continuation claim remains.	Repeat claim-materialisation with the advanced frontier and preserved scope.
Final acceptance	No unresolved lovelace or native assets remain.	Stop; the profile has no continuing claim object.
Rejected transition	Scope, supersession, authorisation, conservation, receipt, continuation, or bounds fail.	No profile acceptance; a valid actor must supply different evidence or a different batch.

2. The domain, network id, Head id, factory id, and claim epoch in the redeemer match the datum.
3. $k > 0$ and $k \leq C.\text{maxOutputCount}$.
4. Every selected output is at or above $C.\text{minAdaFloor}$.
5. Proof size and execution-cost evidence are within the committed bounds.
6. The batch starts strictly after $C.\text{lastSettledRightId}$ in the canonical settled-right order.

7.2 Authorisation

Lerna specifies two authorisation modes. In full authorisation, the settlement evidence must contain exactly the datum-bound signer set. In reduced-signature mode, the authorised signers must be an exact proper subset and the non-interference witness must show that all materialised outputs belong to signed owners and that the untouched frame is preserved. This matters because reduced-signature settlement is a common source of accidental over-claiming: without a frame proof, an output for an unsigned owner could be smuggled into a partial settlement.

7.3 Conservation

Let M be the value materialised by the selected settlement outputs and *Remainder* be the value recorded in the continuation claim, or zero for a final settlement. The transition relation requires:

$$\text{Value}(C) = M + \text{Remainder}.$$

This equality is exact over lovelace and native assets. Native-asset treatment is therefore part of the protocol invariant, not an optional textual extension.

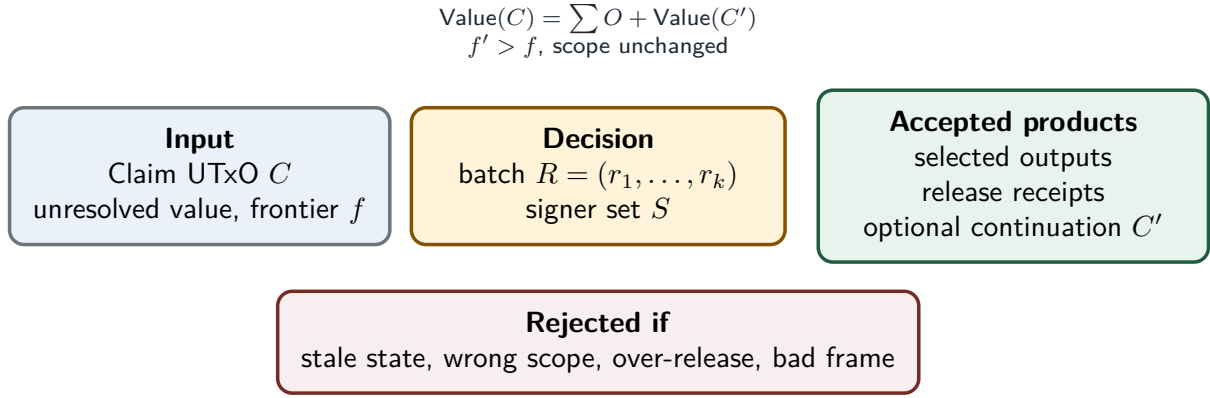


Figure 3: Claim-materialisation as a value-preserving transition. The compact claim is spent once; selected rights become ordinary outputs, released rights produce receipts, and any remaining value is carried by exactly one continuation claim.

7.4 Release Receipts

For each materialised right r_i and selected output o_i , Lerna emits a deterministic receipt:

$$\text{Receipt}_i = \text{H}(\text{Lerna:ClaimUTxO:ReleaseReceipt}, \Sigma, r_i.\text{rightId}, \text{desc}(o_i), i).$$

The transition advances the receipt root from the previous root to a new root covering the settled batch. A later settlement cannot materialise the same right prefix again because the canonical frontier and receipt root must both advance.

7.5 Continuation

If unresolved value remains, exactly one continuing output must return to the claim address with continuing state C' . That state must preserve the scope, signer policy, non-interference root, materialisation budget, proof availability mode, and proof witness root, while advancing the latest-state number, receipt root, settled-right count, last settled right id, unresolved value, and state commitment. If no unresolved value remains, there must be no continuing claim output.

8 Ledger Enforcement

The claim-materialisation profile is intended to be enforced at the ledger boundary by a claim script or equivalent admissibility rule. Its key checks are:

Table 4: Ledger checks required by the claim-materialisation profile.

Check family	Ledger responsibility
Input and continuation	Resolve the own input, compare its value to claim value, require either no continuation for final settlement or exactly one inline-datum continuation.
Identity and domain	Bind datum and redeemer to the same factory, Head, network, epoch, and latest-state domain.
Latest state	Require the old commitment to match the datum and the next state number to be strictly greater.

Table 4: Ledger checks required by the claim-materialisation profile.

Check family	Ledger responsibility
State commitment	Recompute the current and next state commitments from canonical fields.
Receipt progress	Recompute the expected release-receipt root and require it to advance.
Authorisation	Check exact full signers or exact reduced-signature non-interference signers.
Conservation	Require input claim value to equal selected outputs plus continuation value across lovelace and native assets.
Output descriptors	Bind selected output indices, payment key hashes, lovelace, and asset bundles to the redeemer descriptors.
Admission bounds	Enforce positive numeric bounds, output-count bounds, min-ADA policy, and continuation-value safety.

These checks are intentionally redundant with the deterministic transition model in Section 7. The model explains the expected transition; ledger enforcement decides whether a claim spend is admissible; external evidence, where used, should bind the accepted transition to the same scope, value equation, and output descriptors.

9 Acceptance Argument for the Claim-Materialisation Profile

Lemma 9.1 (Scope and supersession). *For the claim-materialisation profile, a settlement transaction cannot validly mix evidence across domains, networks, Heads, factories, or settlement epochs, and cannot settle a state number less than or equal to the datum-bound latest state number.*

Proof. The claim state and transition witness carry the same settlement scope, and ledger enforcement checks equality of the scope fields. The next state number is required to be strictly greater than the datum’s latest-state number, while the current and next commitments are recomputed from canonical fields. $\square$

Lemma 9.2 (Conservation and bounded continuation). *Every accepted claim-materialisation transition conserves the claim input value exactly across selected outputs and, if present, the unique continuation claim.*

Proof. Ledger enforcement compares the input claim value with the sum of selected settlement outputs plus continuation value. The equality is over lovelace and native assets. Admission checks bound the number of selected outputs, proof size, execution units, and min-ADA floor. $\square$

Lemma 9.3 (Receipt frontier). *An accepted transition cannot materialise the same canonical right prefix twice.*

Proof. The transition must start strictly after the previous `lastSettledRightId`, recompute the release-receipt root for the selected batch, and advance both the receipt root and the settled-right frontier. A replayed or reordered prefix fails the frontier or receipt-root check. $\square$

Lemma 9.4 (Reduced-signature non-interference). *In reduced-signature mode, an accepted transition can affect only rights whose owners are represented by the authorised signer subset.*

Proof. The non-interference witness binds the authorised signer subset, touched participants, selected right ids, settlement output descriptors, and the untouched frame. If an unsigned owner’s output is inserted or an untouched right is changed, the witness no longer matches the datum-bound non-interference root. $\square$

Lemma 9.5 (No-stranding). *For every accepted claim-materialisation transition, unresolved value is either paid to selected settlement outputs, preserved in exactly one same-scope continuation claim, or absent because final settlement has reached zero unresolved value.*

Proof. Definition 9.2 gives exact value conservation and requires the continuing value to equal the input claim value minus selected outputs. Section 7.5 requires exactly one continuation when unresolved value remains and no continuation when it does not. Scope preservation and monotone frontier advancement prevent the remaining value from being represented under a different claim identity or a regressed batch boundary. $\square$

Theorem 9.6 (Claim-materialisation safety theorem). *For any accepted claim-materialisation settlement, if the ledger validates the claim spend and the supplied evidence satisfies the transition obligations below, then the settlement:*

1. *spends the script-controlled claim UTxO under the claim admissibility rule;*
2. *binds latest-state evidence to one domain, network id, Head id, factory id, and claim epoch;*
3. *strictly supersedes the previous latest-state number;*
4. *preserves recomputable datum, transition, and continuation commitments;*
5. *uses exact full authorisation or reduced-signature non-interference;*
6. *conserves lovelace and native assets across selected outputs and continuation state;*
7. *emits deterministic release receipts and advances the settled-right frontier;*
8. *leaves no unresolved value stranded outside selected outputs or a same-scope continuation;*
9. *respects committed output-count, proof-size, execution-unit, and min-ADA bounds;*
10. *binds proof availability before materialisation;*
11. *accepts only evidence that is bound to the same scope, value equation, transition, and selected outputs; and*
12. *reaches profile acceptance only when no unresolved claim value remains.*

The theorem should be read as a conditional acceptance theorem for the claim-materialisation profile, not as a free-standing proof of the whole Lerna design. It is intentionally narrow: the claim holds only where the host ledger validates the spend, Definitions 4.4 and 4.5 hold, and the materialised batch remains within the committed budget of Definition 4.6.

Table 5: Acceptance obligations for Definition 9.6.

Obligation	Meaning
Ledger enforcement	Claim spend follows the admissibility rule for the active claim state
Latest-state supersession	Same-scope settlement strictly advances state number
State-commitment integrity	Commitments are recomputable from canonical fields
Exact authorisation	Full or reduced signer evidence matches the datum policy
Non-interference frame	Reduced settlement affects only signed owners
Asset conservation	Lovelace and native assets are exactly conserved
Receipt nullification	Materialised rights cannot be claimed twice
Multi-batch progress	Partial batches advance a canonical frontier
No-stranding	Remaining value is either paid, preserved in one continuation, or absent
Admission bounds	Output count, proof size, costs, and min-ADA are bounded
Proof availability	Witness availability is bound before materialisation
Evidence binding	Ledger observations and transition witnesses bind to the same scope
Final acceptance	Final settlement leaves no unresolved claim value

10 Evidence and Evaluation

The thesis treats instantiation evidence as corroboration, not as the paper’s central claim. The useful evidence question is whether the protocol obligations in Section 9 can be exercised in a concrete ledger environment while preserving the same abstract boundaries: scope binding, state supersession, conservation, authorisation, receipt progress, continuation discipline, and bounded materialisation.

Evidence class	Role in the thesis
Transition examples	Show that final settlement, reduced-signature settlement, native-asset conservation, and partial continuation are all covered by the same state relation.
Negative cases	Exercise stale state, signer mismatch, over-release, receipt mutation, output-descriptor mutation, and frontier regression.
Ledger experiments	Demonstrate that the claim-materialisation profile can be executed against a concrete Cardano/Hydra-style ledger setting.
Independent review	Checks whether the stated assumptions, related-work comparisons, theorem boundary, and review matrix in Sections A and 11 are narrow enough for academic use.

The paper deliberately avoids pinning its main claim to one instantiation. Such details are useful for local reproduction, but they are not the thesis. The thesis is the settlement discipline:

Table 6: Evaluation targets for Lerna instantiations.

Target	Question	Expected evidence
State-machine coverage	Are all states in Table 3 reachable only through admissible transitions?	Positive and negative traces for compact, claim-open, partial, final, and rejected states.
Conservation	Is $\text{Value}(C)$ exactly conserved across selected outputs and continuation?	Lovelace and native-asset cases, including attempted over-release and asset substitution.
Non-interference	Can reduced signer sets affect only signed owners?	Touched right proofs plus rejection of unsigned-owner outputs.
Receipt frontier	Can a settled right prefix be materialised twice?	Receipt root and frontier progression with replay/reordering rejection.
No-stranding	Can unresolved value disappear without either payout or a same-scope continuation?	Partial batches, final batches, and rejected continuation mutations.
Admission bounds	Are output count, proof size, execution cost, and min-ADA limits respected?	Boundary cases around each committed limit.
Assumption audit	Are proof availability and liveness supplied outside the protocol where required?	Deployment documentation for storage, watcher responsibility, and deadlines.

compact rights are acceptable only when the protocol can deterministically materialise them within the stated assumptions.

10.1 Evaluation Targets

The generic evaluation agenda follows the protocol obligations rather than any particular instantiation. Each target asks whether a candidate instantiation preserves the abstract property stated in Definition 9.6.

11 Related Work and Positioning

Section 3 positioned Lerna in the stack. This section states the comparative claim more directly: Lerna is not a replacement for any of the neighbouring protocols; it is an inner rights-materialisation discipline that becomes relevant when those protocols carry compact state.

Table 7 also constrains the novelty claim. Lerna’s contribution is not another Head protocol, virtual-Head transport protocol, or general state-channel framework. Its contribution is the rule that compact factory rights must be repayable by deterministic materialisation before outer settlement machinery is asked to preserve custody.

12 Limitations and Open Questions

The unified profile framing removes status language; it does not remove the safety boundary. Lerna remains bounded in the following ways.

Table 7: Positioning of Lerna against neighbouring work.

Work	Primary object	Lerna boundary
Hydra Head [1, 2]	Off-chain Head ledger with close, contest, and fanout.	Lerna stays below the Head lifecycle and defines how compact rights inside the Head become ordinary ledger claims.
Interhead [5]	Virtual Head movement across physical Heads.	Lerna does not transport Heads; it materialises rights inside a Head or virtual Head context.
Hydrozoa [6]	Cooperative and fallback regimes with deterministic outer rollout.	Lerna borrows the no-stranding and deterministic ordering discipline for inner factory rights, not the full ledger architecture.
eltoo, factories, Perun [7, 8, 9]	Newer-state replacement, shared funding, and adjudication of state-channel evidence.	Lerna adapts the latest-state and factory-locality intuition to Cardano/Hydra eUTxO settlement without importing another dispute game.
Cardano eUTxO [4]	Ledger substrate with explicit outputs, scripts, native assets, and transaction budgets.	Lerna is formulated as a settlement discipline over that substrate: exact value conservation, explicit claims, and bounded materialisation.

1. Instantiation evidence is corroborative; it is not a deployment claim.
2. The theorem assumes Cardano ledger validation and Hydra lifecycle observations behave according to their stated rules.
3. The Head-local profile is safe only under the authorised checkpoint/materialisation assumption stated above. If that assumption is too strong, the application must use claim-materialisation rather than compact Head-local accounting alone.
4. Proof bodies committed by the proof witness root must be available to the verifier.
5. Watcher/operator liveness remains a real deployment assumption.
6. Materialisation is safe only within the committed output-count, proof, execution-unit, and min-ADA budget.
7. Dynamic membership, arbitrary application state, policy-level mint/burn logic, and cross-Head settlement composition are outside the accepted claim-materialisation profile.

12.1 Open Questions

Head-local formalisation.

Can the Head-local profile be given a transition relation and liveness theorem with the same precision as the claim-materialisation profile?

Proof availability.

Which storage and retrieval mechanism should make proof bodies available before materialisation deadlines, and how should non-availability be surfaced to watchers?

Capacity envelopes.

What are the practical output-count, proof-size, execution-cost, and min-ADA envelopes for representative factory sizes on the intended host ledger?

Application semantics.

Which application-level rights can safely be represented by the factory-local non-interference frame without adding an application-specific proof system?

Cross-Head composition.

When an Interhead or Hydrozoa-style host changes the outer settlement context, what extra binding is needed so Lerna’s inner receipts and frontiers remain unambiguous?

13 Conclusion

Lerna’s position is precise when it is described through the safety envelopes in Section 2.4. Interhead virtualises Heads across Heads; Lerna virtualises rights and channels inside a Head or virtual Head. Hydrozoa shows why execution and custody should be separated and why rollout must be deterministic; Lerna applies that ethic to the inner materialisation of compact factory rights. Hydra Head supplies the close/contest/fanout substrate and the practical constraints that make compactness valuable; Lerna repays compactness through deterministic materialisation, release receipts, non-interference checks, and exact value conservation.

Claim-materialisation is therefore one strong settlement profile, not the whole identity of the protocol. The Head-local factory profile remains useful when its liveness assumption is acceptable. The claim-materialisation profile is appropriate when that assumption is too strong and explicit ledger settlement is required. In both profiles, the academic claim is the same: compactness is safe only when enforceable state, deterministic payout order, balance preservation, and non-stranding are made explicit. The remaining work follows directly from Sections 12 and 12.1: formalise the Head-local transition relation, make proof availability operational rather than merely committed, measure real ledger limits, and state every deployment assumption at the same precision as the protocol checks it.

A Protocol Review Matrix

This appendix turns the thesis boundary into a construction-review target. Every row should be readable against Sections 4, 5 and 7 and should have both a positive case and a rejection case in any concrete instantiation.

Table 8: Protocol review matrix for Lerna.

ID	Invariant	Positive case	Rejection case
S1	Settlement scope is unique	Domain, network, Head, factory, and epoch all match Definition 5.1	Transition replays evidence across any scope field
S2	Latest state strictly supersedes	Same-scope state number advances under Definition 9.1	Same-number or lower-number transition is accepted
A1	Authorisation matches policy	Full or reduced signer evidence matches the claim policy in Section 7.2	Extra, missing, or unauthorised signer changes settlement authority

Table 8: Protocol review matrix for Lerna.

ID	Invariant	Positive case	Rejection case
N1	Reduced settlement is non-interfering	Touched rights belong to signed owners and the untouched frame remains committed	Unsigned owner output or untouched right changes during partial settlement
V1	Value is conserved exactly	Selected outputs plus continuation equal the claim value over lovelace and native assets	Over-release, under-release, or asset-type substitution is accepted
R1	Receipts prevent double materialisation	Receipt root and frontier advance for every materialised right	A previously settled right prefix is materialised again
C1	Continuation is unique and bounded	A partial batch creates one continuing claim with advanced frontier and preserved scope	Multiple continuations, wrong scope, or unbounded unresolved value is accepted
B1	Admission bounds are respected	Output count, proof size, execution cost, and min-ADA fit the committed budget	Oversized proof, excessive outputs, or below-floor output is accepted
P1	Proof availability is explicit	Proof witness commitment names material available to the verifier before settlement	Settlement relies on unavailable proof bodies without an assumption or fallback
L1	Liveness assumptions are visible	Watcher/operator deadlines are stated where the protocol cannot force action	The paper implies offline actors can be made to submit evidence

References

- [1] S. Chakravarty, M. D. Coretti, M. Fitzi, P. Gazi, P. Kant, A. Kiayias, and A. Russell. Hydra: Fast Isomorphic State Channels. IACR Cryptology ePrint Archive, Report 2020/299. <https://eprint.iacr.org/2020/299.pdf>.
- [2] Cardano Scaling. Hydra Head protocol documentation. <https://hydra.family/head-protocol/docs/protocol-overview>.
- [3] Cardano Scaling. Hydra known issues and limitations. Consulted 8 June 2026. <https://hydra.family/head-protocol/docs/known-issues>.
- [4] Cardano Documentation. The Extended UTxO accounting model. <https://docs.cardano.org/about-cardano/learn/eutxo-explainer/>.
- [5] M. Jourenko, M. Larangeira, and K. Tanaka. Interhead Hydra: Two Heads are Better than One. IACR Cryptology ePrint Archive, Report 2021/1188. <https://eprint.iacr.org/2021/1188.pdf>.
- [6] G. Flerovsky, I. Rodionov, and P. Dragos. Hydrozoa: Lightweight state channels for Cardano. Technical specification working draft, 7 November 2025. <https://cardano-hydrozoa.github.io/hydrozoa/hydrozoa.pdf>.

- [7] C. Decker, R. Russell, and O. Osuntokun. eltoo: A Simple Layer2 Protocol for Bitcoin. <https://blockstream.com/eltoo.pdf>.
- [8] C. Burchert, C. Decker, and R. Wattenhofer. Scalable Funding of Bitcoin Micropayment Channel Networks. https://tik-old.ee.ethz.ch/file/49218d6b9325ddb4f18aef8830ec567b/1/scalable_funding.pdf.
- [9] S. Dziembowski, S. Faust, and K. Hostakova. General State Channel Networks. ACM CCS 2018.